

TinyWear

Wearable inteligente para
reconocimiento de actividad humana

Curso-taller • ELECTRO 2026

ESP32-S3 + MPU6050 + Wi-Fi directo + UDP unicast + TinyML

Prof. Ernesto Sifuentes de la Hoya

UACJ – Universidad Autónoma de Ciudad Juárez, Chih., México

IIT – Instituto de Ingeniería y Tecnología

Dpto. Ingeniería Eléctrica y Computación



Ruta del curso-taller

Tres días presenciales, un caso integrador y un prototipo que evoluciona en cada sesión.

P1

Día 1- Instrumentar y adquirir

- ❖ Montaje TinyWear
- ❖ MPU6050 e I²C
- ❖ Muestreo a 50 Hz
- ❖ Wi-Fi directo
- ❖ Captura con Python
- ❖ Capturas de prueba

P2

Día 2 - Construir y entrenar

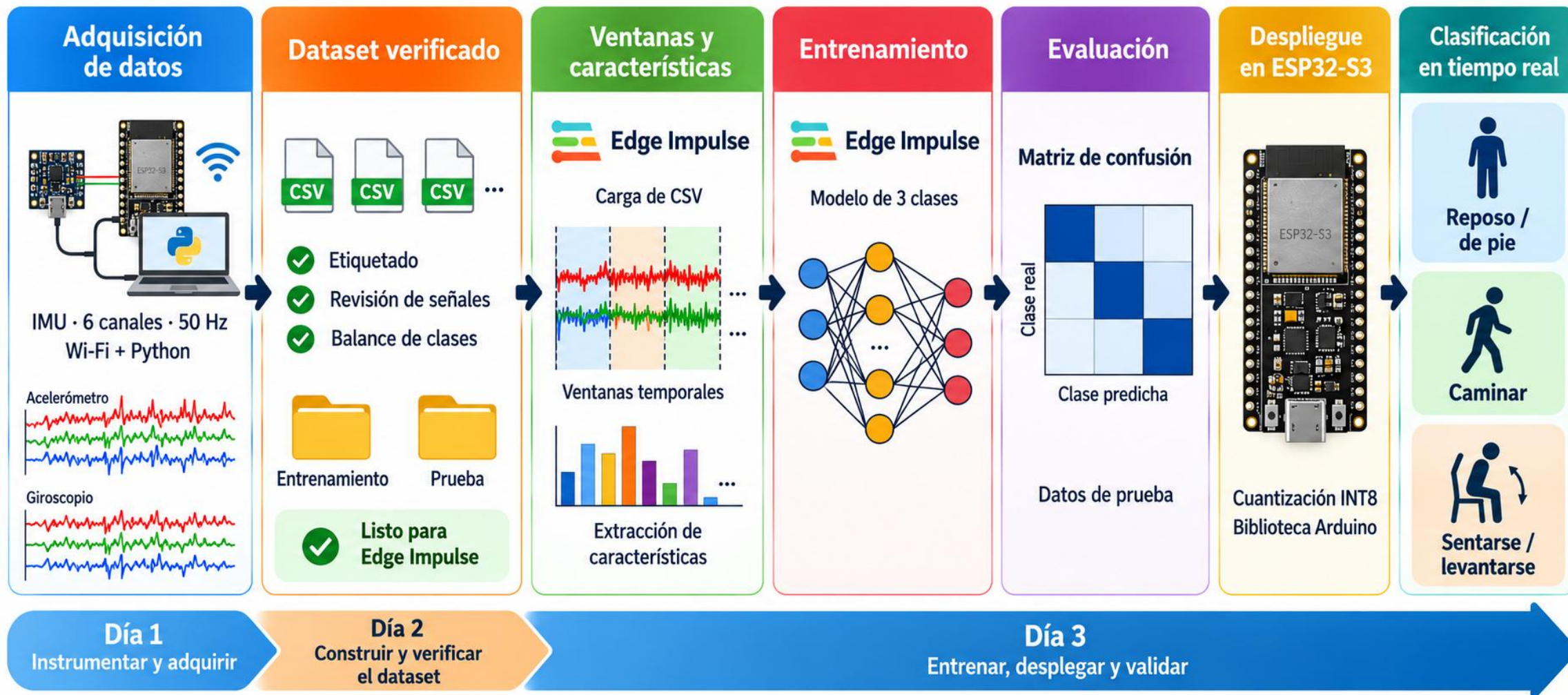
- ❖ Protocolo de las tres actividades
- ❖ Capturas y etiquetado por participante
- ❖ Revisión de señales y Calidad
- ❖ Balance de clases y repeticiones
- ❖ Separación de entrenamiento y prueba
- ❖ Organización y verificación de CSV

Día 3 - Desplegar y validar

- ❖ Carga de CSV en Edge Impulse
- ❖ Ventanas y extracción de características
- ❖ Entrenamiento y matriz de confusión
- ❖ Cuantización y biblioteca Arduino
- ❖ Inferencia en el ESP32-S3
- ❖ Validación y demostración final

Flujo de implementación del TinyWear

Del sensor a la decisión.



Día 2- Construir y entrenar

Meta: dataset limpio, balanceado y trazable.

- ✓ Revisión y limpieza del dataset
- ✓ Ventanas temporales
- ✓ Carga en Edge Impulse
- ✓ Extracción de características
- ✓ Creación y entrenamiento del modelo
- ✓ Evaluar matriz de confusión
- ✓ Validación por sujeto



Protocolo experimental y adquisición de datos

Construir un dataset equilibrado y útil para el entrenamiento de TinyML.



Wearable en la cintura (lado derecho)



Actividades: reposo/de pie, caminar y sentarse/levantarse



Duración nominal por captura: 20 s



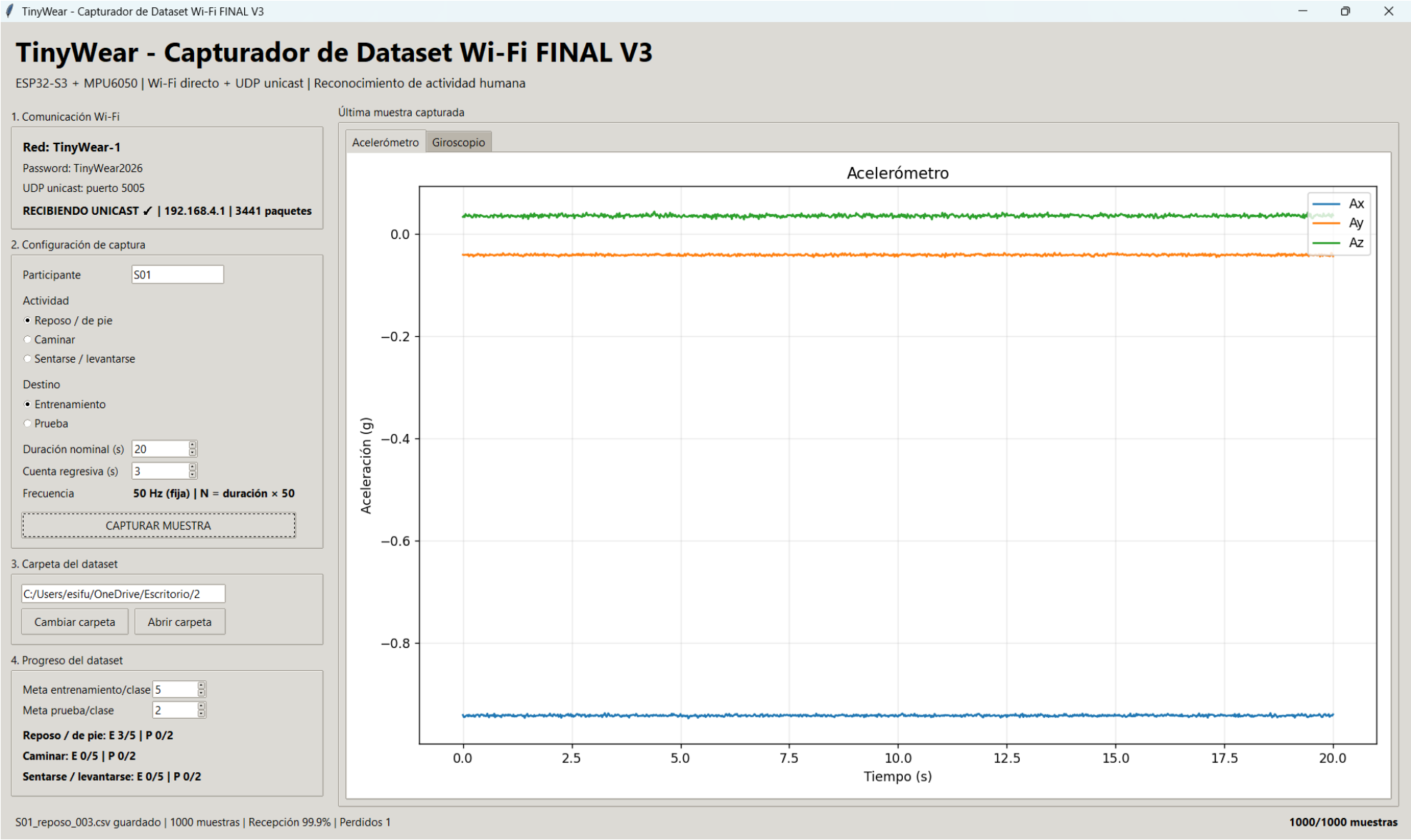
Frecuencia de muestreo: 50 Hz



Separación de muestras para entrenamiento y prueba



Capturador Python



Organización y etiquetado del Dataset

Las muestras deben quedar ordenadas y listas para el entrenamiento.



Participante: S01, S02, ...



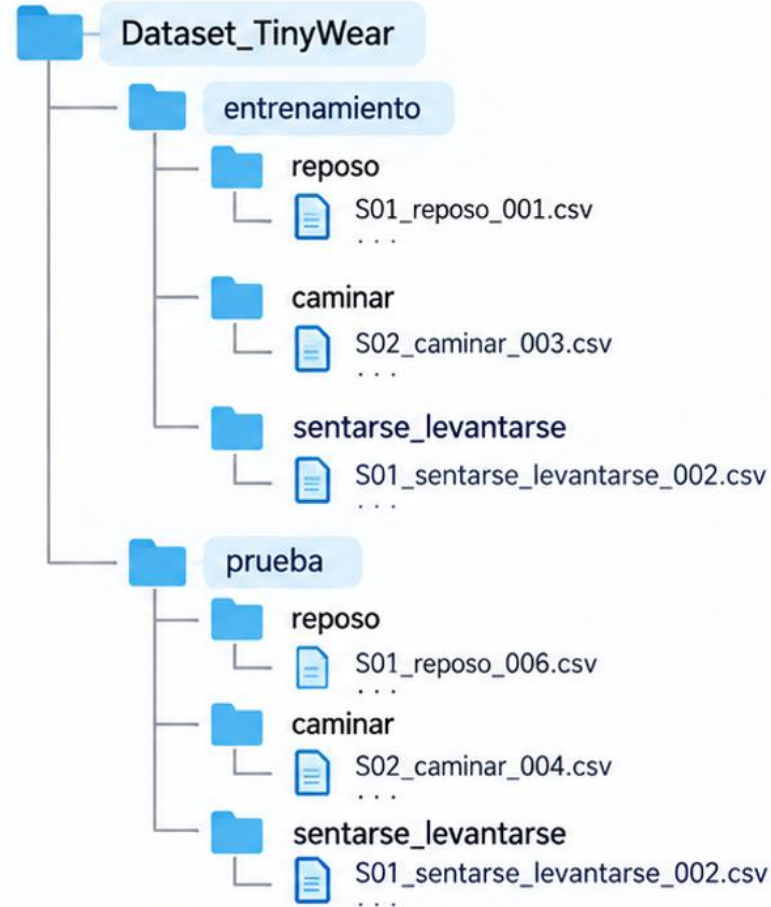
Clases: reposo, caminar, sentarse/levantarse



Separación: entrenamiento y prueba



Un archivo CSV por captura

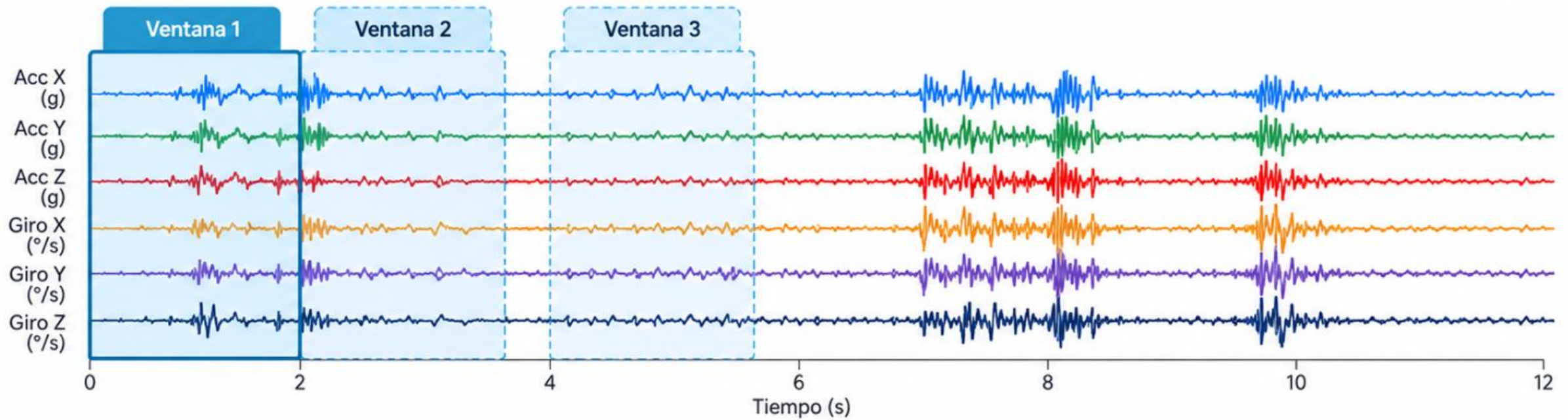


Meta sugerida:

5 capturas de
entrenamiento y
2 de prueba por clase

Ventanas temporales

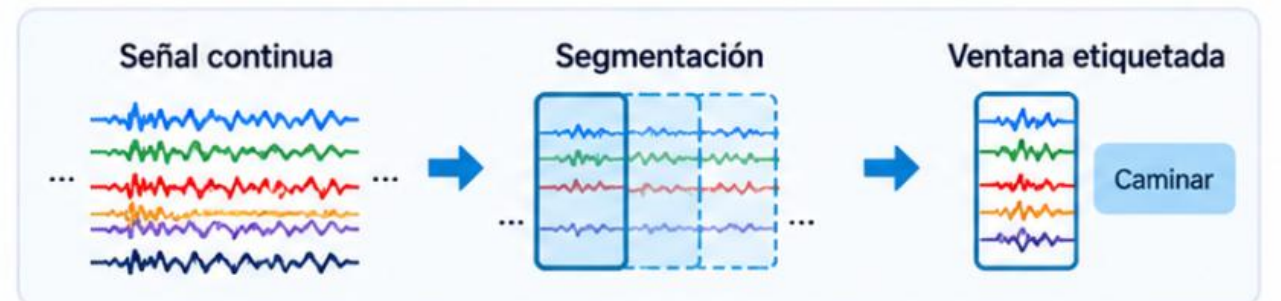
El modelo no clasifica muestras aisladas: clasifica fragmentos de movimiento.



Ventana inicial: 2 s = 100 muestras × 6 canales

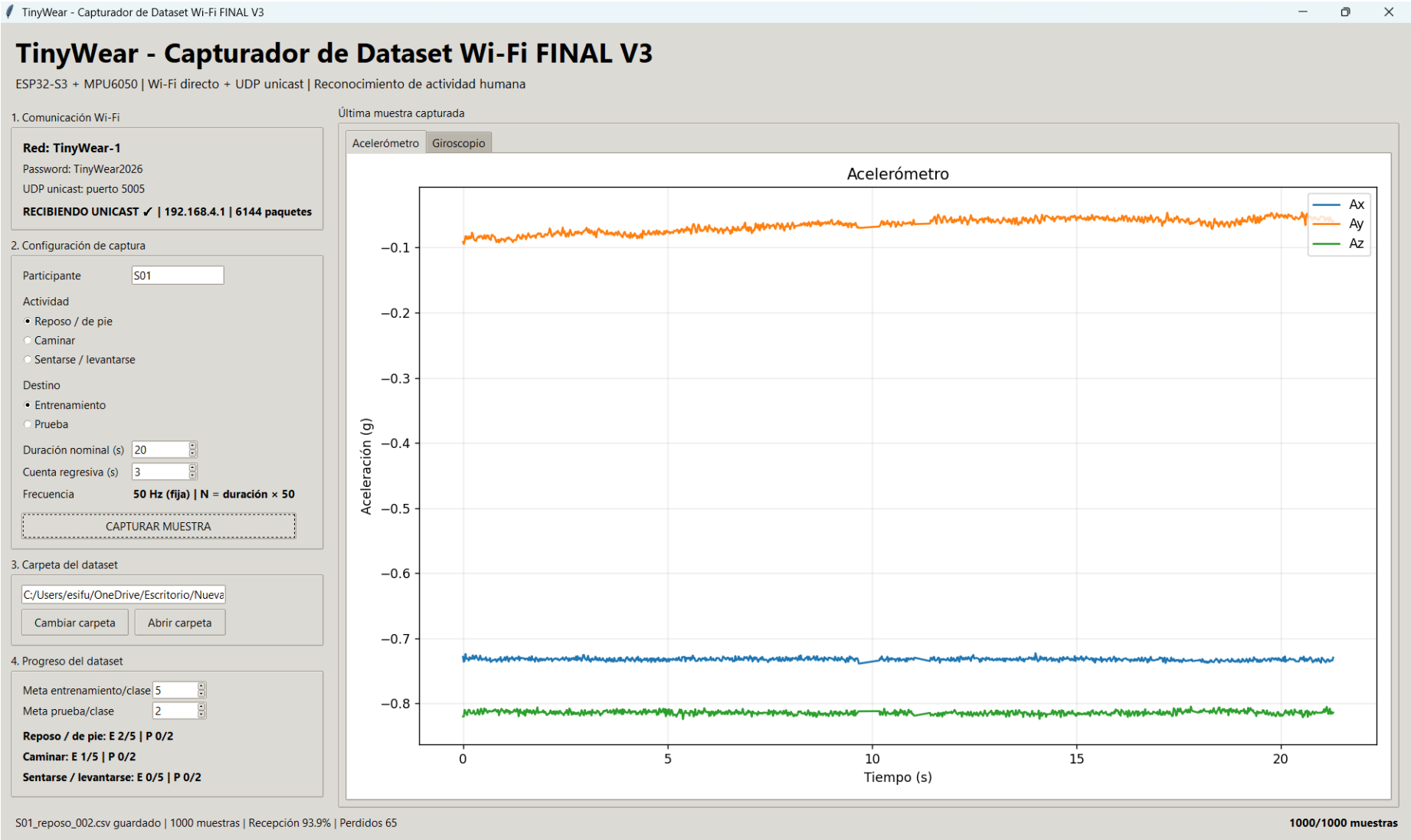


Solapamiento opcional: 50 %



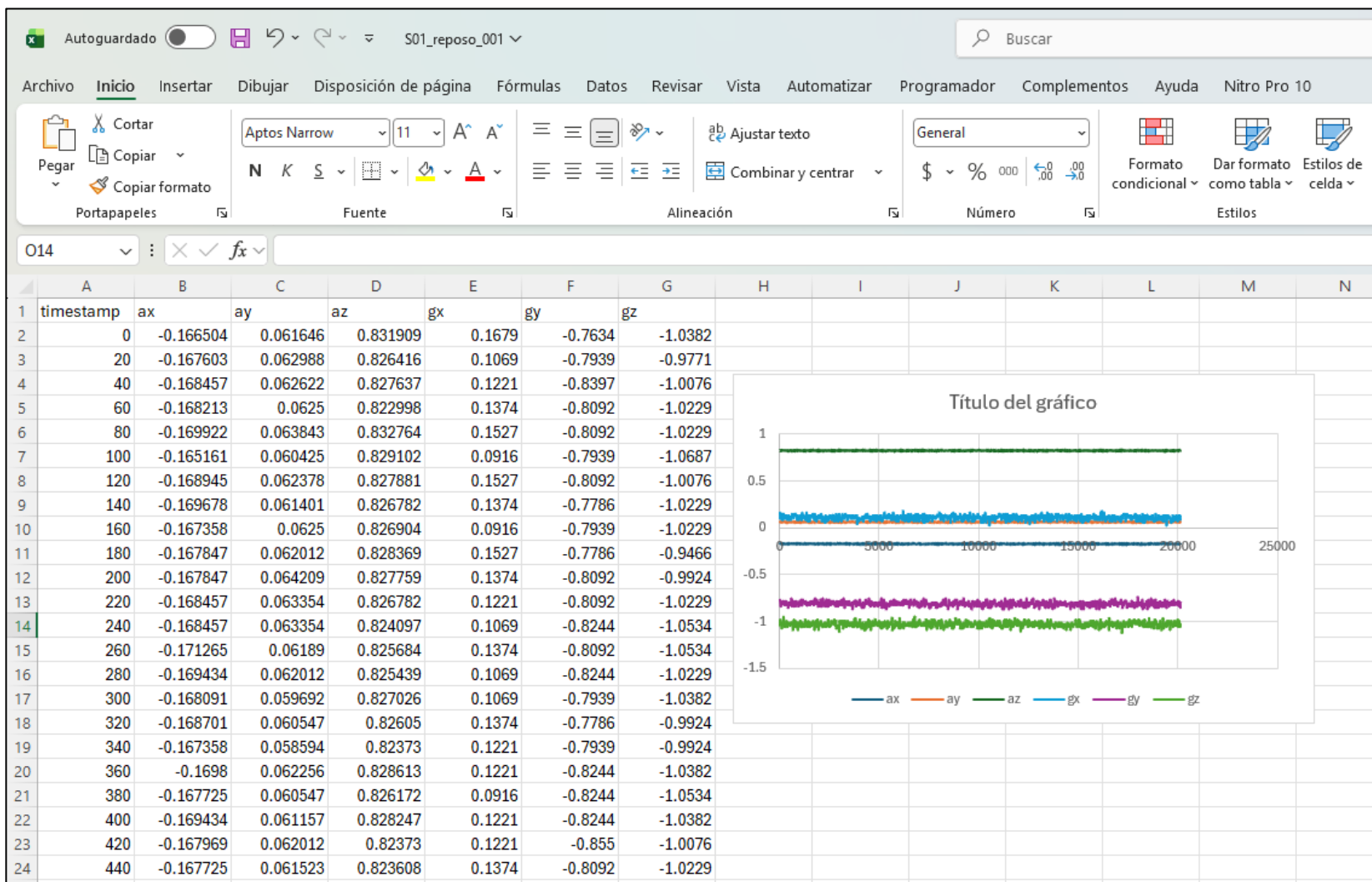
Revisar los archivos (1)

Reposo:



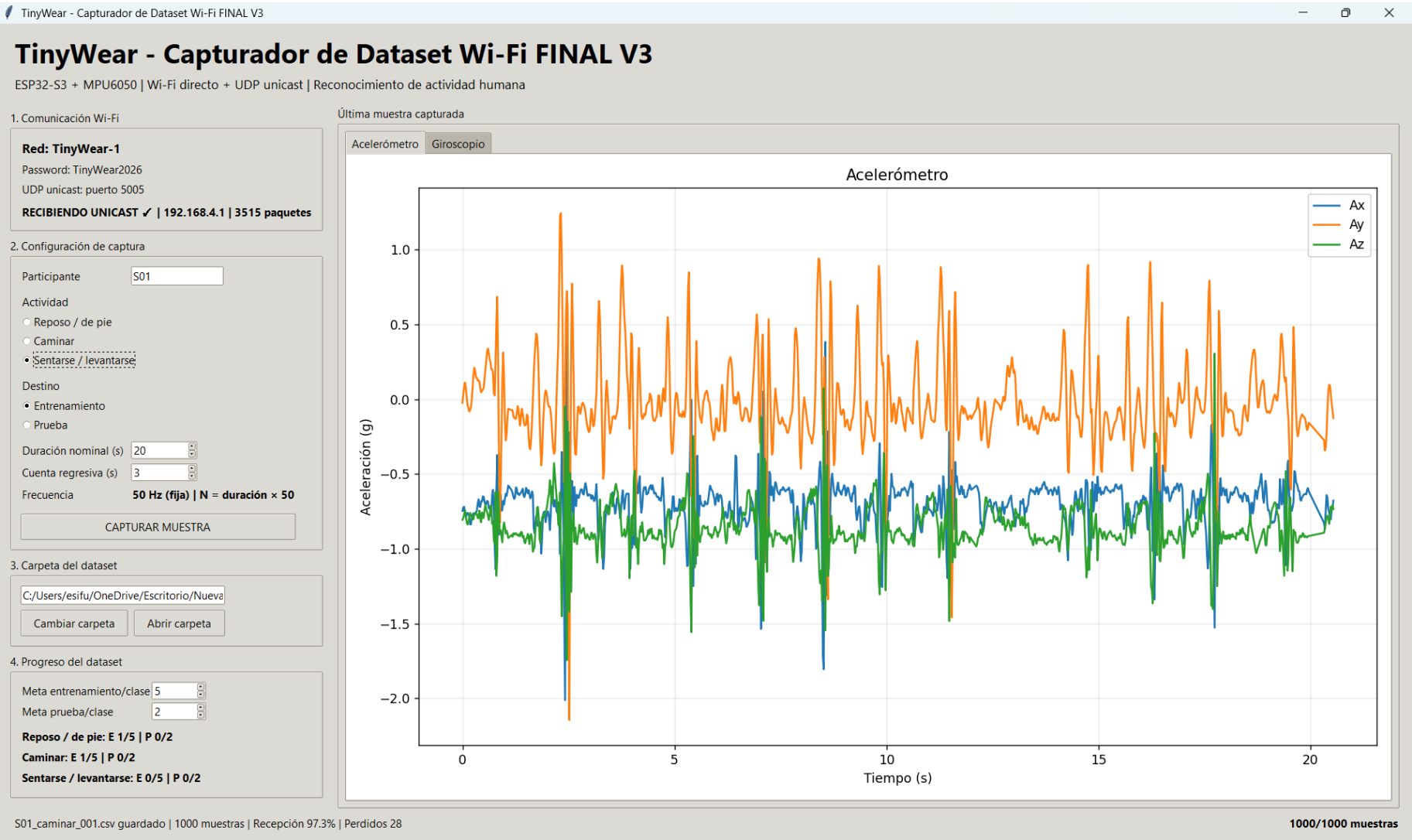
Revisar los archivos (1)

Reposo:



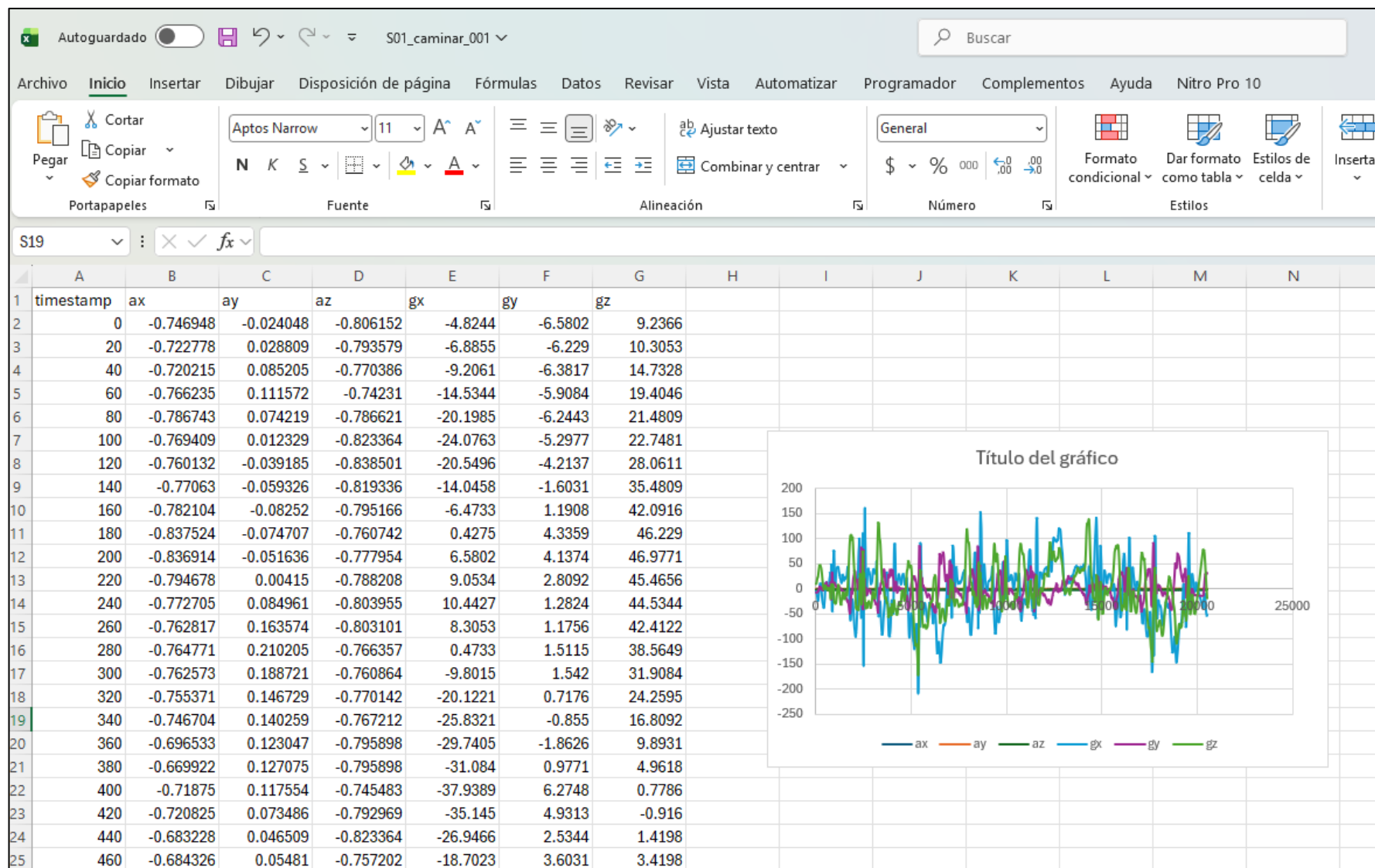
Revisar los archivos (2)

Caminar:



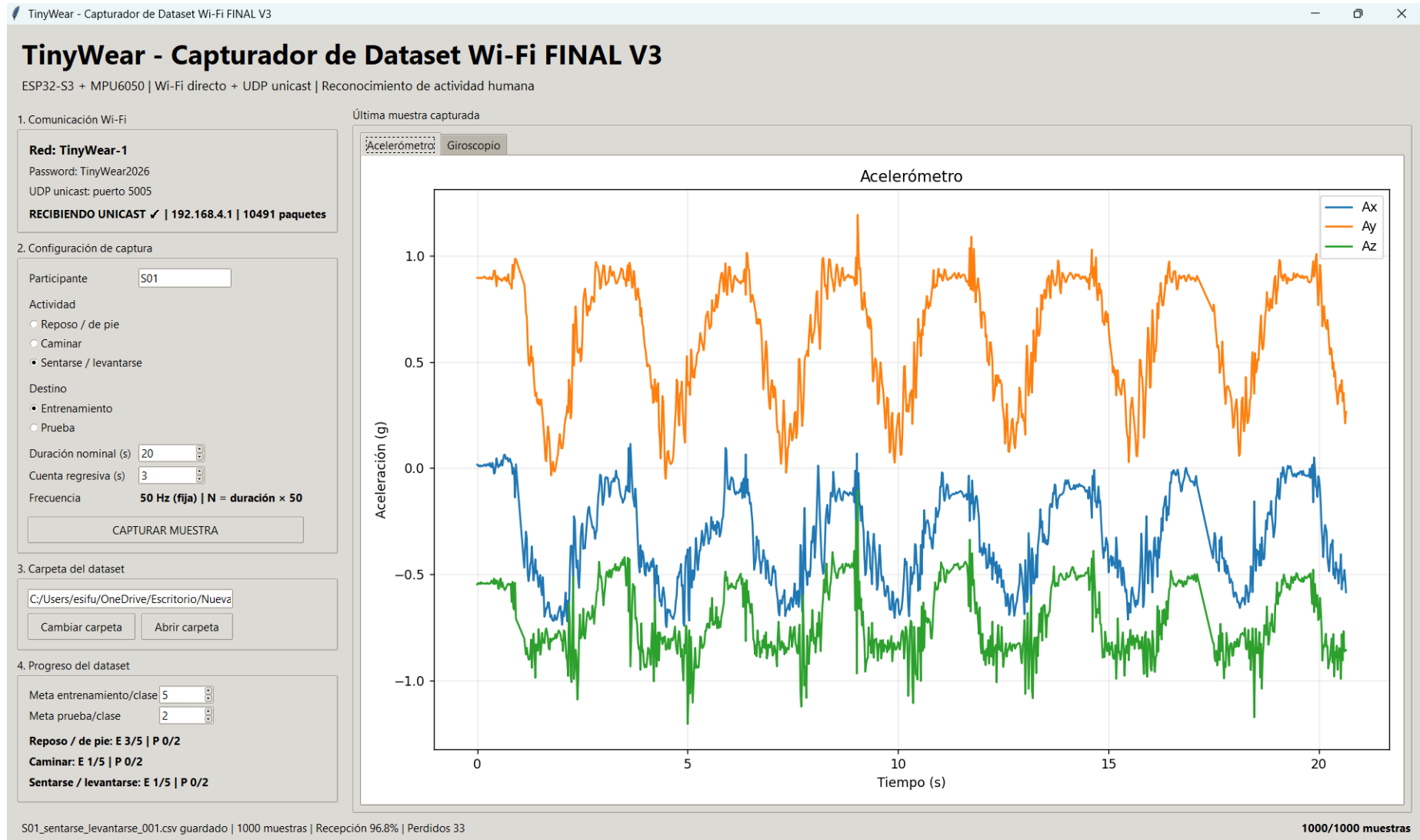
Revisar los archivos (2)

Caminar:



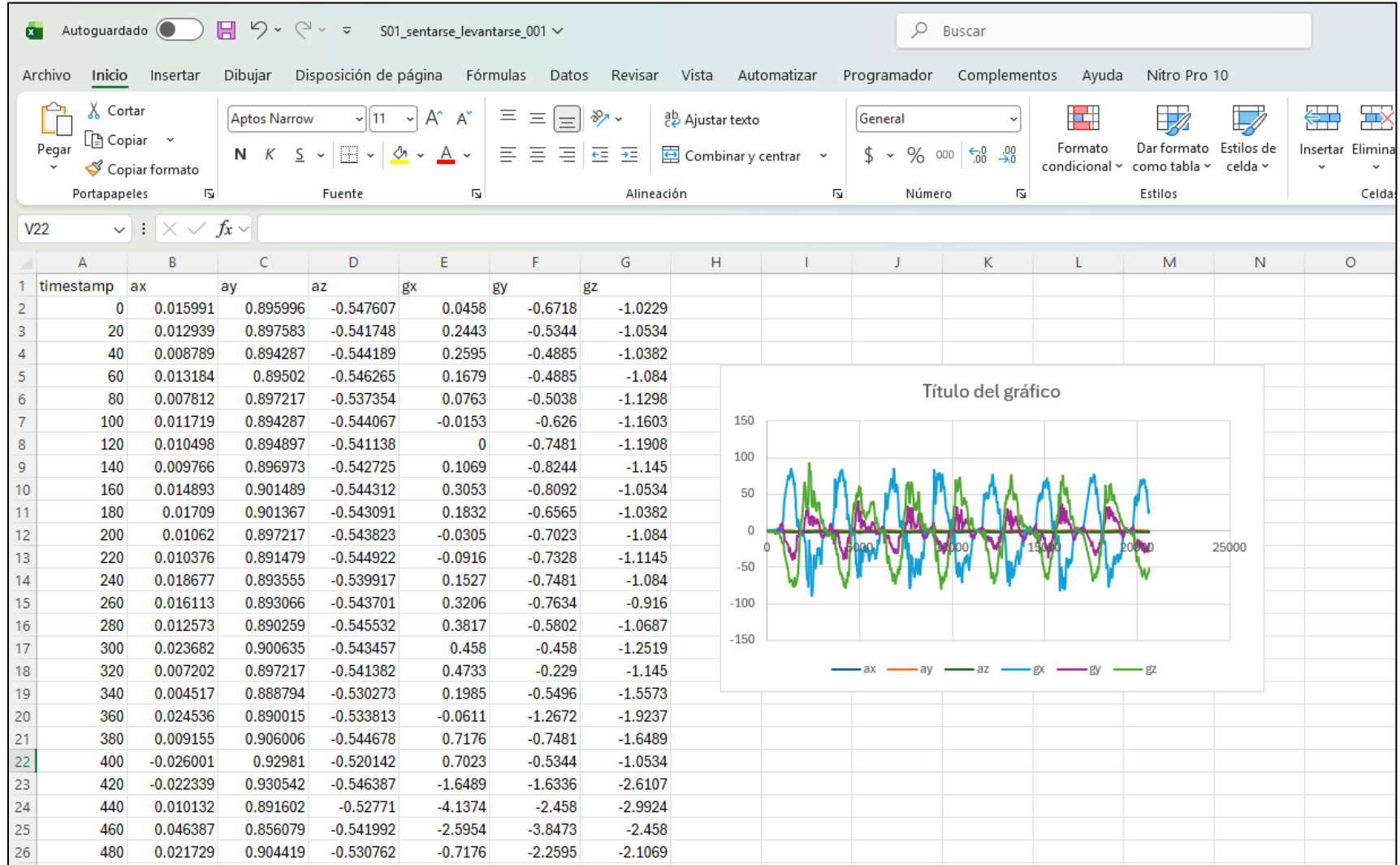
Revisar los archivos (3)

Sentarse/levantarse:



Revisar los archivos (3)

Sentarse/levantarse:



Control de calidad del dataset

Antes de entrenar, las capturas deben revisarse para evitar basura de entrada.

Aceptar una captura cuando...

- ❖ Se recibieron las N muestras esperadas
- ❖ La orientación del wearable no cambió
- ❖ La señal corresponde a la actividad indicada
- ❖ No hay pausas largas ni movimientos accidentales

Repetir una captura cuando...

- ❖ Recepción baja o paquetes perdidos excesivos
- ❖ El participante cambió la posición del sensor
- ❖ La etiqueta no coincide con la actividad real
- ❖ La señal está saturada, plana o incoherente

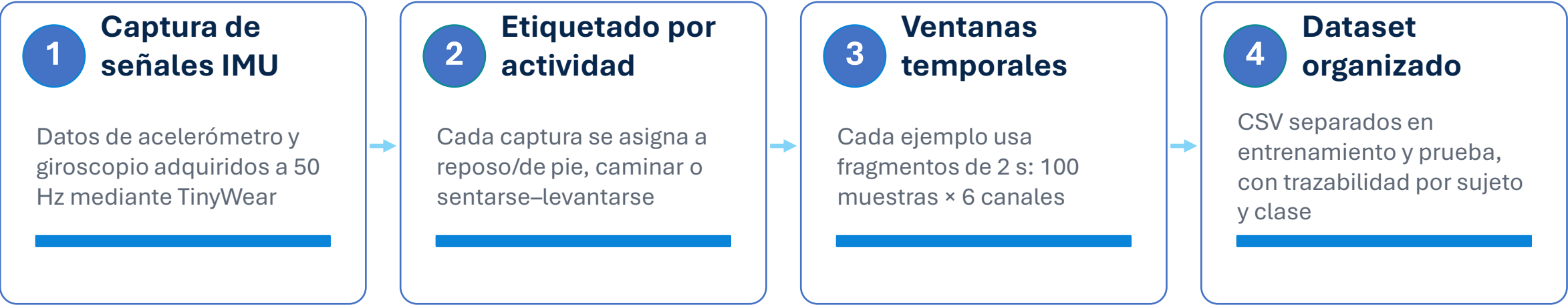
Cierre del día 2

Dataset listo para entrenamiento TinyML.

Etapa	Trabajo práctico	Resultado verificable
Día 1: Instrumentar y adquirir	Montar TinyWear, observar los seis canales y capturar las tres actividades.	CSV etiquetados y revisados.
Día 2: Construir y entrenar	Revisar datos, separar entrenamiento/prueba, configurar ventanas y entrenar en Edge Impulse.	Modelo evaluado y matriz de confusión interpretada.
Día 3: Desplegar y validar	Exportar el modelo, ejecutar inferencia en ESP32-S3 y probar con otro participante.	Prototipo que reconoce actividades y registro de aciertos, errores y tiempo de respuesta.

Cierre del día 2

Dataset listo para entrenamiento TinyML.



Resultado del día:

señales limpias, ventanas definidas y dataset estructurado para entrenar el clasificador

